Golf Green Outliner – Handover Document

📋 Overview

The **Golf Green Outliner** is a browser-based tool that extracts a golf green's boundary from a satellite image and outputs the polygon in **XY world units** for use in SVG, GIS, or the Meshery tool.

🎯 Key Features

Feature	Description
Upload Image	Drag & drop or click to upload a satellite screenshot
Click to Segment	Single click on the green → automatic boundary detection
Drag to Edit	Click and drag any vertex to refine the shape
Tap to Add Points	Click on the polygon line to insert a new vertex
XY World Units	Output in pixels, mm, cm, or inches with adjustable scale
Multiple Formats	Array, SVG Points, or WKT
Export SVG	One-click SVG generation with labels and styling

🧠 Core Methods Explained

1. Flood Fill Segmentation (Color-Based Region Growing)

This is the primary method for finding the green. When you click on the image, the app uses a **flood fill algorithm** to grow a region from the click point.

How It Works:

text

1. Capture the RGB color of the clicked pixel (seed point)

2. Create a stack/queue and push the seed point onto it

3. While the stack is not empty:

   a. Pop a pixel from the stack

   b. Check its four neighbors (up, down, left, right)

   c. For each neighbor:

- Calculate the color distance from the seed color
- If the distance is within tolerance, add it to the region
- Push it onto the stack

4. Continue until no more pixels can be added

Color Distance Formula (Euclidean Distance in RGB Space):

text

distance = $\sqrt{((r2 - r1)^2 + (g2 - g1)^2 + (b2 - b1)^2)}$

Where (r1, g1, b1) is the seed color and (r2, g2, b2) is the candidate pixel color.

Expansion Rounds (Multi-Pass Growth):

After the initial fill, the app runs additional expansion rounds:

text

For each expansion round:

1. Find all edge pixels of the current mask
2. Sample colors from these edge pixels
3. Try to expand outward using these edge colors
4. This captures subtle color variations across the green

Why this works for golf greens:

- Greens have a distinct, relatively uniform color (bright green)
- The tolerance slider controls how much variation is allowed
- The expansion rounds help capture shadows and slight color differences

---

2. Edge Detection (Sobel Operator)

To prevent the flood fill from "bleeding" into the fairway or rough, the app uses **Sobel edge detection** to identify strong boundaries.

How It Works:

Step 1: Convert to Grayscale

text

gray = 0.299 × R + 0.587 × G + 0.114 × B

Step 2: Apply Sobel Kernels

The Sobel operator uses two 3×3 convolution kernels:

Gradient in X direction (Gx):

text

```
┌       ┐
│ -1  0 +1  │
│ -2  0 +2  │
│ -1  0 +1  │
└       ┘
```

Gradient in Y direction (Gy):

text

```
┌       ┐
│ -1  -2  -1  │
│  0   0   0  │
│ +1  +2  +1  │
└       ┘
```

Step 3: Calculate Gradient Magnitude

text

gradient_magnitude = $\sqrt{(Gx^2 + Gy^2)}$

Step 4: Normalize and Threshold

text

edge_strength = gradient_magnitude / max_gradient

The flood fill stops at pixels where edge_strength > edge_threshold.

Why this works for golf greens:

- The boundary between a green and the fairway/rough is often a strong edge
- Even when colors are similar, the texture change creates an edge
- This prevents the fill from spilling into surrounding areas

---

3. Contour Tracing (Moore Neighborhood)

After the flood fill creates a binary mask (1 = green, 0 = not green), the app needs to trace the outer boundary.

How It Works:

Step 1: Find the first edge pixel

text

Scan the image from top-left to bottom-right

Find the first pixel that is:

- Part of the mask (value = 1)
- Has at least one neighbor that is NOT part of the mask

Step 2: Follow the edge using Moore neighborhood

The Moore neighborhood checks all 8 surrounding pixels in a specific order:

text

```
┌─────────┐
│ 7 0 1   │
│ 6 P 2   │ ← P is current pixel
│ 5 4 3   │
└─────────┘
```

Tracing algorithm:

text

1. Start at the first edge pixel
2. Set initial direction to 0 (right)
3. While not back at the start:
   a. Search directions in order: (dir + 0) to (dir + 7) mod 8
   b. For each direction, check if the neighbor pixel is:
      - Part of the mask
      - Also an edge pixel
   c. If found, move to that pixel and update direction

d. Add the pixel to the contour

e. If no edge pixel found, stop

Why this works for golf greens:

- Produces a complete, ordered list of boundary pixels
- Handles complex shapes (coves, peninsulas)
- The resulting contour follows the shape exactly

---

4. Polygon Simplification (Douglas-Peucker Algorithm)

Raw contours can have 100s or even 1000s of points. The Douglas-Peucker algorithm reduces the number of points while preserving the overall shape.

How It Works:

text

```
function simplifyPolygon(points, epsilon):
  if points.length < 3: return points

  // Find the point furthest from the line between first and last
  first = points[0]
  last = points[points.length - 1]
  maxDistance = 0
  maxIndex = 0

  for i from 1 to points.length - 2:
    distance = perpendicularDistance(points[i], first, last)
    if distance > maxDistance:
      maxDistance = distance
      maxIndex = i

  // If the furthest point is far enough, split and recurse
```

```
  if maxDistance > epsilon:
    left = simplifyPolygon(points[0..maxIndex], epsilon)
    right = simplifyPolygon(points[maxIndex..end], epsilon)
    return left + right (without duplicate point)
  else:
    return [first, last]
```

Perpendicular Distance Calculation:

```
perpendicularDistance(point, lineStart, lineEnd):
  dx = lineEnd.x - lineStart.x
  dy = lineEnd.y - lineStart.y
  length = √(dx² + dy²)
  if length == 0:
    return distance from point to lineStart
  return |(point.x - lineStart.x) × dy - (point.y - lineStart.y) × dx| / length
```

Why this works for golf greens:

- Reduces 200+ point contours to ~30-40 key points
- Preserves the essential shape (curves, corners)
- The epsilon parameter controls how aggressive the simplification is

---

5. Curve Smoothing (Catmull-Rom Spline)

Golf greens don't have sharp corners – they're smooth, organic shapes. The Catmull-Rom spline creates a smooth curve that passes through all the simplified points.

How It Works:

A Catmull-Rom spline uses four control points to interpolate between the middle two points.

```
For points P0, P1, P2, P3:
  The curve segment between P1 and P2 is defined by:
```

$$
\begin{aligned}
q(t) = 0.5 \times (\\
&(2 \times P1) + \\
&(-P0 + P2) \times t + \\
&(2 \times P0 - 5 \times P1 + 4 \times P2 - P3) \times t^2 + \\
&(-P0 + 3 \times P1 - 3 \times P2 + P3) \times t^3 \\
)
\end{aligned}
$$

where t goes from 0 to 1

For 2D coordinates, this is applied separately to x and y:

text

$$
\begin{aligned}
x(t) = 0.5 \times (\\
&(2 \times x1) + \\
&(-x0 + x2) \times t + \\
&(2 \times x0 - 5 \times x1 + 4 \times x2 - x3) \times t^2 + \\
&(-x0 + 3 \times x1 - 3 \times x2 + x3) \times t^3 \\
)
\end{aligned}
$$

$$
\begin{aligned}
y(t) = 0.5 \times (\\
&(2 \times y1) + \\
&(-y0 + y2) \times t + \\
&(2 \times y0 - 5 \times y1 + 4 \times y2 - y3) \times t^3 + \\
&(-y0 + 3 \times y1 - 3 \times y2 + y3) \times t^3 \\
)
\end{aligned}
$$

Why this works for golf greens:

- Creates perfectly smooth, organic curves
- Passes through every control point (no approximation)
- The segments parameter controls how smooth the curve is

---

6. Resampling (Evenly Spaced Points)

After smoothing, we might have too many points (e.g., 14 points × 8 segments = 112 points). Resampling evenly spaces points along the perimeter.

How It Works:

text

```
function resampleToTarget(points, targetCount):
  // Calculate total perimeter
  totalLength = 0
  for each segment:
    totalLength += segmentLength

  // Calculate step size
  stepSize = totalLength / targetCount

  // Walk along the perimeter and sample points
  result = []
  distanceTraveled = 0
  currentIndex = 0

  for i from 0 to targetCount - 1:
    targetDistance = i × stepSize

    // Find which segment contains targetDistance
    while distanceTraveled + segmentLength < targetDistance:
      distanceTraveled += segmentLength
      currentIndex++
```

```
    // Interpolate between current and next point
    localT = (targetDistance - distanceTraveled) / segmentLength
    point = interpolate(points[currentIndex], points[currentIndex + 1], localT)
    result.push(point)

  return result
```

Why this works for golf greens:

- Gives exactly the number of points you want
- Points are evenly spaced around the perimeter
- Results in a clean, balanced polygon

---

7. World Unit Conversion

Pixel coordinates are converted to real-world units using a scale factor.

How It Works:

text

X_world = X_pixel × scaleFactor × unitConversionFactor

Y_world = Y_pixel × scaleFactor × unitConversionFactor

Unit Conversion Factors:

Unit	Factor	Description
Pixels	1.0	Raw pixel coordinates
mm	0.264583	Millimeters (1 pixel ≈ 0.264583 mm at 96 DPI)
cm	0.0264583	Centimeters
inches	0.0104167	Inches

Origin Options:

Origin	Transformation
Top-Left	No transformation

Origin	Transformation
Bottom-Left	Y_world = height - Y_pixel
Center	X_world = X_pixel - width/2, Y_world = Y_pixel - height/2

---

8. Interactive Editing (Drag & Tap)

Users can refine the polygon manually after the automatic segmentation.

Drag to Move (Nearest Point Selection):

text

onMouseDown:

  Check all vertices

  Find the closest vertex within 20 pixels

  If found, start dragging

onMouseMove:

  If dragging, update vertex position

  Redraw the polygon

onMouseUp:

  Stop dragging

  Save the new position

Tap to Add Point (Line Intersection):

text

onMouseClick:

  For each segment of the polygon:

    Calculate distance from mouse to the segment

    If distance < 25 pixels:

      Insert a new point at the closest point on the segment

Break the loop

Why this is useful for golf greens:

- Allows fine-tuning of the shape
- Handles cases where automatic detection is slightly off
- No need to restart if the shape isn't perfect

🔄 Complete Processing Pipeline

text

1. User uploads image

↓

2. User clicks on the green

↓

3. Flood Fill (color-based region growing)

- Grows from seed point

- Stops at edges (Sobel detection)

- Multiple expansion rounds

↓

4. Binary Mask

- 1 = green, 0 = not green

↓

5. Contour Tracing (Moore neighborhood)

- Traces the outer boundary

- Returns ordered list of pixels

↓

6. Douglas-Peucker Simplification

- Reduces 100s of points to ~30-40

- Preserves overall shape

↓

7. Catmull-Rom Spline Smoothing

- Creates smooth curves between points
- Passes through all control points

↓

8. Resampling (Evenly Spaced Points)

- Exactly N points (10-25)
- Evenly spaced around perimeter

↓

9. World Unit Conversion

- Pixels → mm/cm/inches
- Adjustable scale and origin

↓

10. Output

- Array, SVG Points, or WKT
- Export to SVG

🖥️ User Interface

Main Controls

Control	Purpose	Range
Tolerance	Color variation allowed	10-100
Expansion	Fill expansion rounds	1-5
Points	Number of output vertices	10-25

Unit Controls

Control	Purpose	Options
Unit Type	Output units	px, mm, cm, inches
Scale	Coordinate multiplier	0.1-100

Control	Purpose	Options
Origin	Coordinate origin	Top-Left, Bottom-Left, Center

Style Controls

Control	Purpose
Line Width	Thickness of polygon outline
Color	Color of polygon and points
Point Size	Size of vertex dots

Output Formats

Format	Use Case
Array	[[x, y], [x, y], ...] – general use
SVG Points	"x,y x,y x,y" – SVG polygon attribute
WKT	"POLYGON((x y, x y, ...))" – GIS/PostGIS

🔧 Troubleshooting

Problem	Solution
No polygon appears	Increase Tolerance (40-60) and click again
Polygon spills into fairway	Decrease Tolerance (20-30) and click again
Polygon is too jagged	Increase Points slider (16-20)
Polygon is too smooth	Decrease Points slider (10-12)
Can't delete a point	Select the point (click it) then click Delete Selected
Image won't load	Check file format (PNG, JPG, JPEG supported)

📤 Output Examples

Array Format (Default)

json

[

[0.0000, 0.0000],

[12.3456, 1.2345],

[23.4567, 5.6789],

[11.1111, 9.9999],

[0.0000, 0.0000]

]

SVG Points Format

text

0.0000,0.0000 12.3456,1.2345 23.4567,5.6789 11.1111,9.9999 0.0000,0.0000

WKT Format

text

POLYGON((0.0000 0.0000, 12.3456 1.2345, 23.4567 5.6789, 11.1111 9.9999, 0.0000 0.0000))

🚀 Integration with Meshery

How to Embed

The app is a **single HTML file** that can be:

1. **Embedded** in an Electron app using <webview> or <iframe>
2. **Served** from a local server
3. **Integrated** as a standalone tool

Data Flow

text

Image → HTML App → Polygon (XY Units) → Meshery Tool

📁 Files to Hand Over

File	Description
green-outliner.html	The complete application (single file)

End of Handover Document

Green-outliner.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Golf Green Outliner – XY World Units</title>
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }
    body {
      font-family: 'Segoe UI', Arial, sans-serif;
      background: #1a2a1f;
      display: flex;
      flex-direction: column;
      align-items: center;
      padding: 20px;
      min-height: 100vh;
    }
```

```
h1 {
  color: #b8d9b0;
  font-weight: 300;
  letter-spacing: 2px;
  margin-bottom: 8px;
}
.sub {
  color: #8aaa82;
  font-size: 14px;
  margin-bottom: 16px;
}
.container {
  background: #2a3d2a;
  border-radius: 16px;
  padding: 20px;
  box-shadow: 0 12px 40px rgba(0, 0, 0, 0.6);
  max-width: 1200px;
  width: 100%;
}
.debug-panel {
  background: #1a2a1a;
  border: 1px solid #3d5a3d;
  border-radius: 8px;
  padding: 10px;
  margin-bottom: 14px;
  max-height: 70px;
  overflow-y: auto;
  font-family: 'Courier New', monospace;
```

```
    font-size: 11px;
    color: #b8d9b0;
}
.debug-panel .error { color: #ff6b6b; }
.debug-panel .success { color: #51cf66; }
.debug-panel .info { color: #74c0fc; }
.debug-panel .warning { color: #ffd43b; }
.toolbar {
    display: flex;
    gap: 10px;
    flex-wrap: wrap;
    margin-bottom: 14px;
    align-items: center;
}
.toolbar input[type="file"] { display: none; }
.toolbar label,
.toolbar button {
    background: #3d5a3d;
    color: #e8f5e0;
    border: none;
    padding: 7px 16px;
    border-radius: 8px;
    font-size: 13px;
    cursor: pointer;
    transition: all 0.2s;
    font-weight: 500;
}
.toolbar label:hover,
```

```
.toolbar button:hover {
    background: #4f704f;
    transform: scale(1.02);
}
.toolbar .status {
    color: #c8e0c0;
    font-size: 13px;
    padding: 5px 12px;
    background: #1f331f;
    border-radius: 20px;
    margin-left: auto;
    white-space: nowrap;
}
.toolbar .point-count {
    color: #8aaa82;
    font-size: 12px;
    padding: 5px 12px;
    background: #1f331f;
    border-radius: 20px;
}
.controls {
    display: flex;
    gap: 14px;
    flex-wrap: wrap;
    background: #1f331f;
    padding: 10px 16px;
    border-radius: 8px;
    margin-bottom: 14px;
```

```
    align-items: center;
}
.controls label {
    color: #b8d9b0;
    font-size: 13px;
    font-weight: 500;
}
.controls input[type="range"] {
    width: 80px;
    accent-color: #4f704f;
}
.controls .value {
    color: #d4f0cc;
    font-size: 13px;
    min-width: 25px;
}
.controls .hint {
    color: #8aaa82;
    font-size: 11px;
    margin-left: 4px;
}
.unit-controls {
    display: flex;
    gap: 14px;
    flex-wrap: wrap;
    background: #1f331f;
    padding: 10px 16px;
    border-radius: 8px;
```

```
    margin-bottom: 14px;
    align-items: center;
    border: 1px solid #3d5a3d;
}
.unit-controls label {
    color: #b8d9b0;
    font-size: 13px;
    font-weight: 500;
}
.unit-controls input[type="number"] {
    background: #2a3d2a;
    border: 1px solid #3d5a3d;
    color: #e8f5e0;
    padding: 4px 8px;
    border-radius: 4px;
    width: 70px;
    font-size: 13px;
}
.unit-controls input[type="number"]:focus {
    outline: 2px solid #4f704f;
}
.unit-controls select {
    background: #2a3d2a;
    border: 1px solid #3d5a3d;
    color: #e8f5e0;
    padding: 4px 8px;
    border-radius: 4px;
    font-size: 13px;
```

```
}
.unit-controls select:focus {
  outline: 2px solid #4f704f;
}
.line-controls {
  display: flex;
  gap: 14px;
  flex-wrap: wrap;
  background: #1f331f;
  padding: 10px 16px;
  border-radius: 8px;
  margin-bottom: 14px;
  align-items: center;
  border: 1px solid #3d5a3d;
}
.line-controls label {
  color: #b8d9b0;
  font-size: 13px;
  font-weight: 500;
}
.line-controls input[type="range"] {
  width: 80px;
  accent-color: #4f704f;
}
.line-controls .value {
  color: #d4f0cc;
  font-size: 13px;
  min-width: 30px;
```

```
}
.line-controls input[type="color"] {
    width: 40px;
    height: 30px;
    padding: 2px;
    border: 1px solid #3d5a3d;
    border-radius: 4px;
    background: #1a2a1a;
    cursor: pointer;
}
.main-layout {
    display: flex;
    gap: 20px;
    flex-wrap: wrap;
}
.map-wrapper {
    flex: 2;
    min-width: 500px;
    background: #1a2a1a;
    border-radius: 12px;
    overflow: hidden;
    border: 2px solid #3d5a3d;
    position: relative;
}
#mapCanvas {
    width: 100%;
    height: auto;
    display: block;
```

```
    cursor: crosshair;
}
.map-overlay {
    position: absolute;
    bottom: 10px;
    left: 50%;
    transform: translateX(-50%);
    background: rgba(0,0,0,0.7);
    color: #b8d9b0;
    padding: 4px 12px;
    border-radius: 8px;
    font-size: 11px;
    pointer-events: none;
    z-index: 5;
}
.coord-display {
    position: absolute;
    top: 10px;
    right: 10px;
    background: rgba(0,0,0,0.7);
    color: #00ff88;
    padding: 4px 10px;
    border-radius: 6px;
    font-family: 'Courier New', monospace;
    font-size: 12px;
    pointer-events: none;
    z-index: 5;
    display: none;
```

```
}
.coord-display.show {
  display: block;
}
.output-panel {
  flex: 1;
  min-width: 280px;
  background: #1f331f;
  border-radius: 12px;
  padding: 16px;
  border: 2px solid #3d5a3d;
  max-height: 600px;
  display: flex;
  flex-direction: column;
}
.output-panel h3 {
  color: #b8d9b0;
  font-weight: 300;
  margin-bottom: 6px;
  font-size: 16px;
}
.output-panel .coord-count {
  color: #8aaa82;
  font-size: 12px;
  margin-bottom: 8px;
}
.output-panel textarea {
  flex: 1;
```

```
    background: #0f1f0f;
    color: #d4f0cc;
    border: 1px solid #3d5a3d;
    border-radius: 8px;
    padding: 10px;
    font-family: 'Courier New', monospace;
    font-size: 11px;
    resize: vertical;
    min-height: 160px;
    width: 100%;
    line-height: 1.5;
}
.output-panel textarea:focus {
    outline: 2px solid #4f704f;
}
.output-panel .actions {
    display: flex;
    gap: 8px;
    margin-top: 8px;
    flex-wrap: wrap;
}
.output-panel .actions button {
    background: #3d5a3d;
    color: #e8f5e0;
    border: none;
    padding: 7px 14px;
    border-radius: 6px;
    cursor: pointer;
```

```
  font-size: 13px;
  flex: 1;
}
.output-panel .actions button:hover {
  background: #4f704f;
}
.instructions {
  color: #9ab892;
  font-size: 13px;
  margin-top: 12px;
  text-align: center;
  background: #1f331f;
  padding: 10px;
  border-radius: 8px;
}
.instructions strong {
  color: #d4f0cc;
}
.instructions .highlight {
  color: #ffd43b;
}
#loadingOverlay {
  display: none;
  position: absolute;
  inset: 0;
  background: rgba(0, 0, 0, 0.6);
  color: white;
  font-size: 20px;
```

```
    justify-content: center;
    align-items: center;
    flex-direction: column;
    gap: 12px;
    border-radius: 12px;
    backdrop-filter: blur(4px);
    z-index: 10;
}
#loadingOverlay .spinner {
    width: 40px;
    height: 40px;
    border: 4px solid #4f704f;
    border-top-color: #b8d9b0;
    border-radius: 50%;
    animation: spin 0.9s linear infinite;
}
@keyframes spin {
    to { transform: rotate(360deg); }
}
.copy-feedback {
    color: #b8d9b0;
    font-size: 12px;
    margin-left: 8px;
    opacity: 0;
    transition: opacity 0.3s;
}
.copy-feedback.show {
    opacity: 1;
```

```
}
.edit-hint {
  color: #ffd43b;
  font-size: 12px;
  text-align: center;
  padding: 4px;
  background: rgba(0,0,0,0.5);
  display: none;
  position: absolute;
  bottom: 30px;
  left: 0;
  right: 0;
  z-index: 5;
}
.edit-hint.show {
  display: block;
}
.format-tabs {
  display: flex;
  gap: 4px;
  margin-bottom: 6px;
}
.format-tabs button {
  background: #1f331f;
  color: #8aaa82;
  border: 1px solid #3d5a3d;
  padding: 3px 10px;
  border-radius: 4px;
```

```
      cursor: pointer;
      font-size: 11px;
      transition: all 0.2s;
    }
    .format-tabs button:hover {
      background: #2a3d2a;
    }
    .format-tabs button.active {
      background: #3d5a3d;
      color: #d4f0cc;
      border-color: #4f704f;
    }
    @media (max-width: 800px) {
      .map-wrapper { min-width: 100%; }
      .output-panel { min-width: 100%; }
    }
  </style>
</head>
<body>

  <h1>⛳ Golf Green Outliner – XY World Units</h1>
  <div class="sub">Upload image → Click green → Get <span class="highlight">XY world units</span> for SVG</div>

  <div class="container">
    <div class="debug-panel" id="debugLog">
      <div class="info">🔍 Ready... Upload a satellite image and click on the green.</div>
```

```
    </div>

    <!-- Unit Controls -->
    <div class="unit-controls">
      <label>📐 World Units:</label>
      <select id="unitType">
        <option value="pixels">Pixels (px)</option>
        <option value="mm">Millimeters (mm)</option>
        <option value="cm">Centimeters (cm)</option>
        <option value="inches">Inches (in)</option>
      </select>

      <label>Scale:</label>
      <input type="number" id="scaleFactor" value="1" step="0.1" min="0.1"
max="100" />
      <span class="hint">(multiplier)</span>

      <label>Origin:</label>
      <select id="originType">
        <option value="top-left">Top-Left (0,0)</option>
        <option value="bottom-left">Bottom-Left (0,0)</option>
        <option value="center">Center</option>
      </select>

      <button id="applyUnitsBtn"
style="background:#4f704f;color:#ffd43b;">Apply</button>
    </div>
```

```
    <div class="line-controls">
      <label>📏 Line:</label>
      <input type="range" id="lineWidthSlider" min="1" max="10" step="0.5" value="3" />
      <span class="value" id="lineWidthValue">3</span>

      <label>🎨 Color:</label>
      <input type="color" id="lineColorPicker" value="#00ff88" />

      <label>⭕ Points:</label>
      <input type="range" id="pointSizeSlider" min="3" max="12" step="1" value="6" />
      <span class="value" id="pointSizeValue">6</span>

      <button id="resetStyleBtn" style="background:#3d5a3d;color:#ffd43b;">↺ Reset</button>
    </div>

    <div class="controls">
      <label>Tolerance:</label>
      <input type="range" id="toleranceSlider" min="10" max="100" value="35" />
      <span class="value" id="toleranceValue">35</span>
      <span class="hint">(higher = more inclusive)</span>

      <label>Expansion:</label>
      <input type="range" id="expansionSlider" min="1" max="5" step="1" value="3" />
      <span class="value" id="expansionValue">3</span>

      <label>Points:</label>
```

```
    <input type="range" id="vertexTarget" min="10" max="20" step="1" value="14" />
    <span class="value" id="vertexValue">14</span>
  </div>

  <div class="toolbar">
    <label for="imageUpload" style="background:#4f704f;color:#ffd43b;">📤 Upload Image</label>
    <input type="file" id="imageUpload" accept="image/*" />
    <button id="resetBtn">↺ Reset</button>
    <button id="deletePointBtn">➖ Delete Selected</button>
    <button id="undoBtn">↩ Undo</button>
    <button id="exportSVGBtn" style="background:#4f704f;color:#ffd43b;">📐 Export SVG</button>
    <span class="point-count" id="pointCount">Points: 0</span>
    <span class="status" id="statusText">Ready</span>
  </div>

  <div class="main-layout">
    <div class="map-wrapper">
      <canvas id="mapCanvas"></canvas>
      <div class="coord-display" id="coordDisplay">📍 0, 0</div>
      <div id="loadingOverlay">
        <div class="spinner"></div>
        <span>Processing...</span>
      </div>
      <div class="edit-hint" id="editHint">🖍️ <span class="highlight">Click the green</span> → <span class="highlight">Drag dots</span> to edit → <span class="highlight">Tap line</span> to add points</div>
      <div class="map-overlay" id="mapOverlay">📤 Upload an image to start</div>
```

```
    </div>

    <div class="output-panel">
      <h3>📐 Polygon (XY World Units)</h3>
      <div class="coord-count" id="coordCount">0 vertices</div>
      <div class="format-tabs">
        <button class="active" data-format="array">Array</button>
        <button data-format="svg">SVG Points</button>
        <button data-format="wkt">WKT</button>
      </div>
      <textarea id="outputText" readonly></textarea>
      <div class="actions">
        <button id="copyBtn">📋 Copy</button>
        <button id="downloadBtn">💾 Download</button>
        <span class="copy-feedback" id="copyFeedback">✅ Copied!</span>
      </div>
      <div class="comparison">
        <div class="stat">Vertices: <span id="finalCount">0</span></div>
        <div class="stat">Area: <span id="areaDisplay">0</span> px²</div>
      </div>
    </div>
  </div>

  <div class="instructions">
    <strong>1.</strong> Upload a satellite image (screenshot)  | 
    <strong>2.</strong> Click <strong>inside</strong> the green  | 
    <strong>3.</strong> Adjust <span class="highlight">Units & Scale</span> above  | 
```

```
        <strong>4.</strong> Copy <span class="highlight">XY coordinates</span> for your SVG
      </div>
    </div>

    <script>
      // ============================================================
      // XY WORLD UNITS VERSION – SVG-friendly coordinates
      // ============================================================

      // DOM refs
      const canvas = document.getElementById('mapCanvas');
      const ctx = canvas.getContext('2d');
      const fileInput = document.getElementById('imageUpload');
      const coordDisplay = document.getElementById('coordDisplay');

      const resetBtn = document.getElementById('resetBtn');
      const deletePointBtn = document.getElementById('deletePointBtn');
      const undoBtn = document.getElementById('undoBtn');
      const exportSVGBtn = document.getElementById('exportSVGBtn');
      const statusEl = document.getElementById('statusText');
      const pointCountEl = document.getElementById('pointCount');
      const overlay = document.getElementById('loadingOverlay');
      const outputText = document.getElementById('outputText');
      const coordCount = document.getElementById('coordCount');
      const copyBtn = document.getElementById('copyBtn');
      const downloadBtn = document.getElementById('downloadBtn');
      const copyFeedback = document.getElementById('copyFeedback');
```

```
const debugLog = document.getElementById('debugLog');
const editHint = document.getElementById('editHint');
const mapOverlay = document.getElementById('mapOverlay');

const toleranceSlider = document.getElementById('toleranceSlider');
const toleranceValue = document.getElementById('toleranceValue');
const expansionSlider = document.getElementById('expansionSlider');
const expansionValue = document.getElementById('expansionValue');
const vertexTarget = document.getElementById('vertexTarget');
const vertexValue = document.getElementById('vertexValue');
const finalCount = document.getElementById('finalCount');
const areaDisplay = document.getElementById('areaDisplay');

// Unit controls
const unitType = document.getElementById('unitType');
const scaleFactor = document.getElementById('scaleFactor');
const originType = document.getElementById('originType');
const applyUnitsBtn = document.getElementById('applyUnitsBtn');

// Style controls
const lineWidthSlider = document.getElementById('lineWidthSlider');
const lineWidthValue = document.getElementById('lineWidthValue');
const lineColorPicker = document.getElementById('lineColorPicker');
const pointSizeSlider = document.getElementById('pointSizeSlider');
const pointSizeValue = document.getElementById('pointSizeValue');
const resetStyleBtn = document.getElementById('resetStyleBtn');

// Format tabs
```

```
const formatTabs = document.querySelectorAll('.format-tabs button');
let currentFormat = 'array';

let img = null;
let imageData = null;
let polygonPixels = [];
let polygonWorld = [];
let history = [];
let isProcessing = false;
let isDragging = false;
let dragIndex = -1;
let selectedIndex = -1;

// Style state
let lineWidth = 3;
let lineColor = '#00ff88';
let pointSize = 6;

// Unit state
let currentUnit = 'pixels';
let currentScale = 1;
let currentOrigin = 'top-left';

// ---------- Debug ----------
function log(msg, type = 'info') {
  const div = document.createElement('div');
  div.className = type;
  div.textContent = `[${new Date().toLocaleTimeString()}] ${msg}`;
```

```
    debugLog.appendChild(div);
    debugLog.scrollTop = debugLog.scrollHeight;
    console.log(`[${type}]`, msg);
}

// ---------- Unit conversion ----------
function convertToWorldUnits(pixelPoints, width, height) {
    const unit = currentUnit;
    const scale = parseFloat(scaleFactor.value) || 1;
    const origin = currentOrigin;

    // Base conversion factors (approximate for SVG)
    const unitFactors = {
        'pixels': 1,
        'mm': 0.264583,
        'cm': 0.0264583,
        'inches': 0.0104167
    };

    const factor = unitFactors[unit] || 1;

    let result = pixelPoints.map(([x, y]) => {
        let wx = x * factor * scale;
        let wy = y * factor * scale;

        // Apply origin
        if (origin === 'bottom-left') {
            wy = height * factor * scale - wy;
```

```
    } else if (origin === 'center') {
      wx = wx - (width / 2) * factor * scale;
      wy = wy - (height / 2) * factor * scale;
    }

    return [wx, wy];
  });

  return result;
}

function getUnitLabel() {
  const labels = {
    'pixels': 'px',
    'mm': 'mm',
    'cm': 'cm',
    'inches': 'in'
  };
  return labels[currentUnit] || 'px';
}

// ---------- Format output ----------
function formatOutput(points, format) {
  const label = getUnitLabel();

  switch (format) {
    case 'svg':
      // SVG points attribute format: "x1,y1 x2,y2 x3,y3 ..."
```

```
            return points.map(([x, y]) => `${x.toFixed(4)},${y.toFixed(4)}`).join(' ');

          case 'wkt':
            // Well-Known Text: "POLYGON((x1 y1, x2 y2, ...))"
            const pts = points.map(([x, y]) => `${x.toFixed(4)} ${y.toFixed(4)}`).join(', ');
            return `POLYGON((${pts}, ${points[0][0].toFixed(4)} ${points[0][1].toFixed(4)}))`;

          case 'array':
          default:
            // JSON array: [[x1, y1], [x2, y2], ...]
            return JSON.stringify(points, null, 2);
        }
      }

      // ---------- Update output ----------
      function updateOutput() {
        if (polygonPixels.length < 3 || !img) {
          outputText.value = '';
          coordCount.textContent = '0 vertices';
          finalCount.textContent = '0';
          areaDisplay.textContent = '0';
          return;
        }

        const w = canvas.width;
        const h = canvas.height;
```

```
    // Convert to world units
    const worldPoints = convertToWorldUnits(polygonPixels, w, h);
    const label = getUnitLabel();

    const formatted = formatOutput(worldPoints, currentFormat);
    outputText.value = formatted;

    coordCount.textContent = `${worldPoints.length} vertices (${label})`;
    finalCount.textContent = worldPoints.length;

    // Calculate area in world units
    let area = 0;
    for (let i = 0; i < worldPoints.length - 1; i++) {
      area += worldPoints[i][0] * worldPoints[i + 1][1];
      area -= worldPoints[i + 1][0] * worldPoints[i][1];
    }
    area = Math.abs(area / 2);
    areaDisplay.textContent = area.toFixed(2) + ' ' + label + '²';
}

// ---------- Format tabs ----------
formatTabs.forEach(tab => {
    tab.addEventListener('click', function() {
      formatTabs.forEach(t => t.classList.remove('active'));
      this.classList.add('active');
      currentFormat = this.dataset.format;
      updateOutput();
    });
```

```
});

// ---------- Apply units ----------
applyUnitsBtn.addEventListener('click', function() {
  currentUnit = unitType.value;
  currentOrigin = originType.value;
  updateOutput();
  log(`📐 Units: ${currentUnit}, Origin: ${currentOrigin}, Scale: ${scaleFactor.value}`, 'info');
  statusEl.textContent = `✅ Units updated: ${currentUnit} | Origin: ${currentOrigin}`;
});

// Also update on change
unitType.addEventListener('change', () => { applyUnitsBtn.click(); });
originType.addEventListener('change', () => { applyUnitsBtn.click(); });
scaleFactor.addEventListener('change', () => { applyUnitsBtn.click(); });

// ---------- Style controls ----------
lineWidthSlider.addEventListener('input', () => {
  lineWidth = parseFloat(lineWidthSlider.value);
  lineWidthValue.textContent = lineWidth.toFixed(1);
  draw();
});

lineColorPicker.addEventListener('input', () => {
  lineColor = lineColorPicker.value;
  draw();
```

```
});

pointSizeSlider.addEventListener('input', () => {
  pointSize = parseInt(pointSizeSlider.value);
  pointSizeValue.textContent = pointSize;
  draw();
});

resetStyleBtn.addEventListener('click', () => {
  lineWidthSlider.value = 3;
  lineWidth = 3;
  lineWidthValue.textContent = '3';
  lineColorPicker.value = '#00ff88';
  lineColor = '#00ff88';
  pointSizeSlider.value = 6;
  pointSize = 6;
  pointSizeValue.textContent = '6';
  draw();
  log('↺ Style reset', 'info');
  statusEl.textContent = '↺ Style reset to default';
});

// ---------- Slider displays ----------
toleranceSlider.addEventListener('input', () => { toleranceValue.textContent = toleranceSlider.value; });
expansionSlider.addEventListener('input', () => { expansionValue.textContent = expansionSlider.value; });
vertexTarget.addEventListener('input', () => { vertexValue.textContent = vertexTarget.value; });
```

```
// ---------- Load image ----------
function loadImage(file) {
  log(`📂 Loading: ${file.name}`, 'info');
  statusEl.textContent = '📂 Loading image...';

  const reader = new FileReader();
  reader.onload = (e) => {
    const image = new Image();
    image.onload = () => {
      img = image;
      canvas.width = image.width;
      canvas.height = image.height;
      canvas.style.width = '100%';
      canvas.style.height = 'auto';

      const tempCanvas = document.createElement('canvas');
      tempCanvas.width = image.width;
      tempCanvas.height = image.height;
      const tempCtx = tempCanvas.getContext('2d');
      tempCtx.drawImage(image, 0, 0);
      imageData = tempCtx.getImageData(0, 0, image.width, image.height);

      ctx.clearRect(0, 0, canvas.width, canvas.height);
      ctx.drawImage(img, 0, 0);
      window.img = img;

      statusEl.textContent = '✅ Image loaded – click inside the green';
```

```
        log(` ✅ ${image.width}x${image.height}px`, 'success');
        mapOverlay.textContent = '📸 Click the green!';

        polygonPixels = [];
        polygonWorld = [];
        history = [];
        selectedIndex = -1;
        updateOutput();
        pointCountEl.textContent = 'Points: 0';
        editHint.classList.add('show');

        canvas.scrollIntoView({ behavior: 'smooth', block: 'center' });
      };
      image.onerror = (err) => {
        log(` ❌ Image load error: ${err}`, 'error');
        statusEl.textContent = '❌ Failed to load image';
      };
      image.src = e.target.result;
    };
    reader.onerror = (err) => {
      log(` ❌ File read error: ${err}`, 'error');
      statusEl.textContent = '❌ Failed to read file';
    };
    reader.readAsDataURL(file);
}

// ---------- Undo ----------
```

```
function saveHistory() {
  if (polygonPixels.length > 0) {
    history.push(JSON.stringify(polygonPixels));
    if (history.length > 50) history.shift();
  }
}

function undo() {
  if (history.length === 0) {
    statusEl.textContent = '⚠️ Nothing to undo';
    return;
  }
  const last = history.pop();
  polygonPixels = JSON.parse(last);
  selectedIndex = -1;
  draw();
  updateOutput();
  statusEl.textContent = `↩ Undo (${polygonPixels.length} points)`;
  log('↩ Undo', 'info');
}

// ---------- Draw ----------
function draw() {
  if (!window.img) return;
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  ctx.drawImage(window.img, 0, 0);

  if (polygonPixels.length > 2) {
```

```
// Draw filled polygon
ctx.beginPath();
ctx.moveTo(polygonPixels[0][0], polygonPixels[0][1]);
for (let i = 1; i < polygonPixels.length; i++) {
  ctx.lineTo(polygonPixels[i][0], polygonPixels[i][1]);
}
ctx.closePath();
ctx.fillStyle = 'rgba(0, 255, 136, 0.06)';
ctx.fill();

// Draw polygon outline
ctx.beginPath();
ctx.moveTo(polygonPixels[0][0], polygonPixels[0][1]);
for (let i = 1; i < polygonPixels.length; i++) {
  ctx.lineTo(polygonPixels[i][0], polygonPixels[i][1]);
}
ctx.closePath();
ctx.strokeStyle = lineColor;
ctx.lineWidth = lineWidth;
ctx.shadowColor = lineColor + '44';
ctx.shadowBlur = 8;
ctx.stroke();
ctx.shadowBlur = 0;

// Draw vertices
for (let i = 0; i < polygonPixels.length; i++) {
  const p = polygonPixels[i];
  const isSelected = (i === selectedIndex);
```

```
        const radius = isSelected ? pointSize + 3 : pointSize;

        ctx.beginPath();
        ctx.arc(p[0], p[1], radius, 0, 2 * Math.PI);
        ctx.fillStyle = isSelected ? '#ffd43b' : lineColor;
        ctx.shadowBlur = isSelected ? 16 : 8;
        ctx.shadowColor = isSelected ? '#ffd43b' : lineColor + '66';
        ctx.fill();
        ctx.shadowBlur = 0;

        // Number label
        if (polygonPixels.length <= 30) {
          ctx.fillStyle = '#1a2a1f';
          ctx.font = `bold ${Math.max(6, Math.min(10, pointSize - 1))}px sans-serif`;
          ctx.textAlign = 'center';
          ctx.textBaseline = 'middle';
          ctx.fillText(i, p[0], p[1] + 0.5);
        }
      }
    }

    pointCountEl.textContent = `Points: ${polygonPixels.length}`;
}

// ---------- Color distance ----------
function colorDist(c1, c2) {
    return Math.sqrt((c1.r - c2.r) ** 2 + (c1.g - c2.g) ** 2 + (c1.b - c2.b) ** 2);
}
```

```
// ---------- Flood fill ----------
function floodFill(sx, sy, tolerance, rounds) {
  if (!imageData) return null;
  const w = imageData.width,
    h = imageData.height,
    data = imageData.data;
  const base = {
    r: data[(sy * w + sx) * 4],
    g: data[(sy * w + sx) * 4 + 1],
    b: data[(sy * w + sx) * 4 + 2]
  };
  log(` 🎯 Click at (${sx}, ${sy}) RGB(${base.r},${base.g},${base.b})` , 'info');

  let mask = new Uint8Array(w * h);
  let visited = new Uint8Array(w * h);
  let stack = [
    [sx, sy]
  ];
  visited[sy * w + sx] = 1;
  mask[sy * w + sx] = 1;

  while (stack.length > 0) {
    const [x, y] = stack.pop();
    const neighbors = [
      [x - 1, y],
      [x + 1, y],
      [x, y - 1],
```

```
      [x, y + 1]
    ];
    for (const [nx, ny] of neighbors) {
      if (nx < 0 || nx >= w || ny < 0 || ny >= h) continue;
      const idx = ny * w + nx;
      if (visited[idx]) continue;
      const c = {
        r: data[idx * 4],
        g: data[idx * 4 + 1],
        b: data[idx * 4 + 2]
      };
      if (colorDist(base, c) <= tolerance) {
        visited[idx] = 1;
        mask[idx] = 1;
        stack.push([nx, ny]);
      } else {
        visited[idx] = 1;
      }
    }
  }

  for (let r = 0; r < rounds; r++) {
    const edgePixels = [];
    for (let y = 0; y < h; y++) {
      for (let x = 0; x < w; x++) {
        const idx = y * w + x;
        if (mask[idx] === 1) {
          const neighbors = [
```

```
            [-1, 0],
            [1, 0],
            [0, -1],
            [0, 1]
          ];
          for (const [dx, dy] of neighbors) {
            const nx = x + dx,
              ny = y + dy;
            if (nx < 0 || nx >= w || ny < 0 || ny >= h) { edgePixels.push([x, y]); break; }
            if (mask[ny * w + nx] === 0) { edgePixels.push([x, y]); break; }
          }
        }
      }
    }
    if (edgePixels.length === 0) break;

    const newPixels = [];
    const step = Math.max(1, Math.floor(edgePixels.length / 15));
    for (let i = 0; i < edgePixels.length; i += step) {
      const [x, y] = edgePixels[i];
      const idx = y * w + x;
      const edgeColor = {
        r: data[idx * 4],
        g: data[idx * 4 + 1],
        b: data[idx * 4 + 2]
      };
      const neighbors = [
        [x - 1, y],
```

```
        [x + 1, y],
        [x, y - 1],
        [x, y + 1]
      ];
      for (const [nx, ny] of neighbors) {
        if (nx < 0 || nx >= w || ny < 0 || ny >= h) continue;
        const nidx = ny * w + nx;
        if (mask[nidx] === 1) continue;
        const c = {
          r: data[nidx * 4],
          g: data[nidx * 4 + 1],
          b: data[nidx * 4 + 2]
        };
        if (colorDist(edgeColor, c) <= tolerance * 1.3) {
          newPixels.push([nx, ny]);
        }
      }
    }
    let added = 0;
    for (const [nx, ny] of newPixels) {
      const idx = ny * w + nx;
      if (mask[idx] === 0) { mask[idx] = 1;
        added++; }
    }
    if (added < 30) break;
  }

  let count = 0;
```

```
  for (let i = 0; i < w * h; i++)
    if (mask[i] === 1) count++;
  log(` 📊 Mask: ${count} pixels `, 'info');
  return count > 50 ? mask : null;
}

// ---------- Contour tracing ----------
function traceContour(mask) {
  if (!mask) return [];
  const w = imageData.width,
    h = imageData.height;
  let sx = -1,
    sy = -1;
  for (let y = 0; y < h; y++) {
    for (let x = 0; x < w; x++) {
      const idx = y * w + x;
      if (mask[idx] === 1) {
        const neighbors = [
          [-1, -1],
          [-1, 0],
          [-1, 1],
          [0, -1],
          [0, 1],
          [1, -1],
          [1, 0],
          [1, 1]
        ];
        let isEdge = false;
```

```
        for (const [dx, dy] of neighbors) {
          const nx = x + dx,
            ny = y + dy;
          if (nx < 0 || nx >= w || ny < 0 || ny >= h) { isEdge = true; break; }
          if (mask[ny * w + nx] === 0) { isEdge = true; break; }
        }
        if (isEdge) { sx = x;
          sy = y; break; }
      }
    }
    if (sx >= 0) break;
}
if (sx < 0) return [];

const contour = [];
let cx = sx,
    cy = sy,
    dir = 0;
const dirs = [
    [1, 0],
    [1, 1],
    [0, 1],
    [-1, 1],
    [-1, 0],
    [-1, -1],
    [0, -1],
    [1, -1]
];
```

```
let iter = 0;
do {
  contour.push([cx, cy]);
  let found = false;
  for (let i = 0; i < 8; i++) {
    const nd = (dir + i) % 8;
    const [dx, dy] = dirs[nd];
    const nx = cx + dx,
      ny = cy + dy;
    if (nx >= 0 && nx < w && ny >= 0 && ny < h && mask[ny * w + nx] === 1) {
      let isEdge = false;
      const neighbors = [
        [-1, -1],
        [-1, 0],
        [-1, 1],
        [0, -1],
        [0, 1],
        [1, -1],
        [1, 0],
        [1, 1]
      ];
      for (const [dx2, dy2] of neighbors) {
        const nx2 = nx + dx2,
          ny2 = ny + dy2;
        if (nx2 < 0 || nx2 >= w || ny2 < 0 || ny2 >= h) { isEdge = true; break; }
        if (mask[ny2 * w + nx2] === 0) { isEdge = true; break; }
      }
      if (isEdge) { cx = nx;
```

```
                cy = ny;
                dir = (nd + 5) % 8;
                found = true; break; }
            }
        }
        if (!found) break;
        iter++;
        if (iter > 50000) break;
    } while (cx !== sx || cy !== sy);

    log(` 🔍 Contour: ${contour.length} points`, 'info');
    return contour;
}

// ---------- SMOOTHING ----------

function simplifyPolygon(points, eps) {
    if (points.length < 3) return points;
    let maxD = 0,
        idx = 0;
    const first = points[0],
        last = points[points.length - 1];
    for (let i = 1; i < points.length - 1; i++) {
        const d = perpDist(points[i], first, last);
        if (d > maxD) { maxD = d;
            idx = i; }
    }
    if (maxD > eps) {
```

```
      const left = simplifyPolygon(points.slice(0, idx + 1), eps);
      const right = simplifyPolygon(points.slice(idx), eps);
      return left.slice(0, -1).concat(right);
    }
    return [first, last];
}

function perpDist(p, a, b) {
    const dx = b[0] - a[0],
      dy = b[1] - a[1];
    const len = Math.sqrt(dx * dx + dy * dy);
    if (len === 0) return Math.sqrt((p[0] - a[0]) ** 2 + (p[1] - a[1]) ** 2);
    return Math.abs((p[0] - a[0]) * dy - (p[1] - a[1]) * dx) / len;
}

function catmullRomToPolygon(points, segs) {
    if (points.length < 3) return points;
    const result = [];
    const n = points.length;
    for (let i = 0; i < n; i++) {
      const p0 = points[(i - 1 + n) % n];
      const p1 = points[i];
      const p2 = points[(i + 1) % n];
      const p3 = points[(i + 2) % n];
      for (let t = 0; t <= 1; t += 1 / segs) {
        const t2 = t * t,
          t3 = t2 * t;
```

```
            const x = 0.5 * ((2 * p1[0]) + (-p0[0] + p2[0]) * t + (2 * p0[0] - 5 * p1[0] + 4 * p2[0] -
p3[0]) * t2 +
                (-p0[0] + 3 * p1[0] - 3 * p2[0] + p3[0]) * t3);
            const y = 0.5 * ((2 * p1[1]) + (-p0[1] + p2[1]) * t + (2 * p0[1] - 5 * p1[1] + 4 * p2[1] -
p3[1]) * t2 +
                (-p0[1] + 3 * p1[1] - 3 * p2[1] + p3[1]) * t3);
            if (i === 0 && t === 0) continue;
            result.push([x, y]);
          }
        }
        return result;
    }

    function getCenter(points) {
        let cx = 0,
            cy = 0;
        for (const p of points) {
            cx += p[0];
            cy += p[1];
        }
        return [cx / points.length, cy / points.length];
    }

    function sortByAngle(points, center) {
        return points.slice().sort((a, b) => {
            return Math.atan2(a[1] - center[1], a[0] - center[0]) -
                Math.atan2(b[1] - center[1], b[0] - center[0]);
        });
    }
```

```
function resampleToTarget(points, target) {
  if (points.length < 3 || target < 3) return points;

  let totalLen = 0;
  const segLens = [];
  for (let i = 0; i < points.length; i++) {
    const p1 = points[i];
    const p2 = points[(i + 1) % points.length];
    const dx = p2[0] - p1[0];
    const dy = p2[1] - p1[1];
    const len = Math.sqrt(dx * dx + dy * dy);
    segLens.push(len);
    totalLen += len;
  }

  if (totalLen < 1) return points;

  const result = [];
  const step = totalLen / target;

  for (let i = 0; i < target; i++) {
    const targetDist = i * step;
    let dist = 0;
    let idx = 0;
    while (idx < segLens.length && dist + segLens[idx] < targetDist) {
      dist += segLens[idx];
      idx++;
```

```
      }
      if (idx >= segLens.length) idx = segLens.length - 1;

      const localDist = targetDist - dist;
      const segLen = segLens[idx];
      const t = segLen > 0 ? localDist / segLen : 0;
      const p1 = points[idx];
      const p2 = points[(idx + 1) % points.length];

      const x = p1[0] + (p2[0] - p1[0]) * t;
      const y = p1[1] + (p2[1] - p1[1]) * t;
      result.push([x, y]);
    }

    if (result.length > 2) {
      const first = result[0],
        last = result[result.length - 1];
      if (first[0] !== last[0] || first[1] !== last[1]) {
        result.push([...first]);
      }
    }

    return result;
}

function createSmoothGreen(raw, target) {
    if (raw.length < 4) return raw;
```

```
let poly = [...raw];

poly = simplifyPolygon(poly, 3.0);
log(`📐 After simplify: ${poly.length}`, 'info');

const center = getCenter(poly);
poly = sortByAngle(poly, center);
log(`📐 After sorting: ${poly.length}`, 'info');

poly = catmullRomToPolygon(poly, 4);
log(`📐 After Catmull-Rom: ${poly.length}`, 'info');

poly = simplifyPolygon(poly, 1.0);
log(`📐 After simplify again: ${poly.length}`, 'info');

poly = resampleToTarget(poly, target);
log(`📐 After resample: ${poly.length} (target: ${target})`, 'info');

if (poly.length > 2) {
  const first = poly[0],
    last = poly[poly.length - 1];
  if (first[0] !== last[0] || first[1] !== last[1]) {
    poly.push([...first]);
  }
}

log(`📐 Final: ${poly.length} vertices`, 'success');
```

```
  return poly;
}

// ---------- MAIN SEGMENTATION ----------
function segmentPoint(x, y) {
  if (isProcessing) return;
  if (!imageData) {
    statusEl.textContent = '⚠️ Upload an image first';
    log('❌ No image data', 'error');
    return;
  }

  log(`🎯 Click at (${Math.round(x)}, ${Math.round(y)})`, 'info');
  isProcessing = true;
  overlay.style.display = 'flex';
  statusEl.textContent = '⏳ Processing...';

  setTimeout(() => {
    try {
      const tol = parseInt(toleranceSlider.value);
      const rounds = parseInt(expansionSlider.value);
      const target = Math.min(parseInt(vertexTarget.value), 25);

      const mask = floodFill(Math.round(x), Math.round(y), tol, rounds);
      if (!mask) {
        statusEl.textContent = '❌ No region found – try increasing Tolerance';
        log('❌ No region found', 'error');
```

```
            overlay.style.display = 'none';
            isProcessing = false;
            return;
        }

        const raw = traceContour(mask);
        if (raw.length < 10) {
            statusEl.textContent = '❌ Contour too small – try increasing Tolerance';
            log('❌ Contour too small', 'error');
            overlay.style.display = 'none';
            isProcessing = false;
            return;
        }

        polygonPixels = createSmoothGreen(raw, target);
        saveHistory();

        selectedIndex = -1;
        draw();
        updateOutput();
        statusEl.textContent = `✅ Green outlined (${polygonPixels.length} points) – XY world units ready`;
        log(`🎉 Done! ${polygonPixels.length} points`, 'success');

        // Auto-apply units
        applyUnitsBtn.click();

    } catch (e) {
```

```
        log(` ❌ ${e.message}` , 'error');
        console.error(e);
        statusEl.textContent = '❌ Error – see console';
      }
      overlay.style.display = 'none';
      isProcessing = false;
    }, 100);
}

// ---------- TAP LINE TO ADD POINT ----------
function findClosestPointOnLine(mx, my) {
  if (polygonPixels.length < 3) return -1;

  let minDist = 25;
  let bestIdx = -1;

  for (let i = 0; i < polygonPixels.length; i++) {
    const p1 = polygonPixels[i];
    const p2 = polygonPixels[(i + 1) % polygonPixels.length];

    const dx = p2[0] - p1[0];
    const dy = p2[1] - p1[1];
    const len2 = dx * dx + dy * dy;

    let t = ((mx - p1[0]) * dx + (my - p1[1]) * dy) / len2;
    t = Math.max(0, Math.min(1, t));

    const projX = p1[0] + t * dx;
```

```
      const projY = p1[1] + t * dy;

      const dist = Math.sqrt((mx - projX) ** 2 + (my - projY) ** 2);

      if (dist < minDist) {
        minDist = dist;
        bestIdx = i;
      }
    }

    return bestIdx;
}

function addPointOnLine(edgeIdx) {
    if (polygonPixels.length < 3 || edgeIdx < 0) return;

    saveHistory();

    const p1 = polygonPixels[edgeIdx];
    const p2 = polygonPixels[(edgeIdx + 1) % polygonPixels.length];

    const midX = (p1[0] + p2[0]) / 2;
    const midY = (p1[1] + p2[1]) / 2;

    polygonPixels.splice(edgeIdx + 1, 0, [midX, midY]);
    draw();
    updateOutput();
    statusEl.textContent = `➕ Added point (${polygonPixels.length} total)`;
```

```
    log(` ➕ Added point on line`, 'info');
}

// ---------- EDITING ----------

function findClosestPoint(mx, my, threshold = 20) {
    let minDist = threshold;
    let minIdx = -1;
    for (let i = 0; i < polygonPixels.length; i++) {
        const p = polygonPixels[i];
        const dist = Math.sqrt((p[0] - mx) ** 2 + (p[1] - my) ** 2);
        if (dist < minDist) {
            minDist = dist;
            minIdx = i;
        }
    }
    return minIdx;
}

// ---------- Mouse events ----------
canvas.addEventListener('mousemove', function(e) {
    const rect = canvas.getBoundingClientRect();
    const scaleX = canvas.width / rect.width;
    const scaleY = canvas.height / rect.height;
    const mx = (e.clientX - rect.left) * scaleX;
    const my = (e.clientY - rect.top) * scaleY;

    coordDisplay.textContent = `📍 ${Math.round(mx)}, ${Math.round(my)}`;
```

```
  coordDisplay.classList.add('show');

  if (polygonPixels.length > 0) {
    const idx = findClosestPoint(mx, my);
    if (idx >= 0) {
      canvas.style.cursor = 'grab';
      return;
    }
    const edgeIdx = findClosestPointOnLine(mx, my);
    canvas.style.cursor = edgeIdx >= 0 ? 'pointer' : 'crosshair';
  } else {
    canvas.style.cursor = 'crosshair';
  }
});

canvas.addEventListener('mouseleave', function() {
  coordDisplay.classList.remove('show');
});

canvas.addEventListener('mousedown', function(e) {
  const rect = canvas.getBoundingClientRect();
  const scaleX = canvas.width / rect.width;
  const scaleY = canvas.height / rect.height;
  const mx = (e.clientX - rect.left) * scaleX;
  const my = (e.clientY - rect.top) * scaleY;

  if (polygonPixels.length === 0) {
    segmentPoint(mx, my);
```

```
      return;
    }

    const idx = findClosestPoint(mx, my);
    if (idx >= 0) {
      isDragging = true;
      dragIndex = idx;
      selectedIndex = idx;
      canvas.style.cursor = 'grabbing';
      draw();
      return;
    }

    const edgeIdx = findClosestPointOnLine(mx, my);
    if (edgeIdx >= 0) {
      addPointOnLine(edgeIdx);
      return;
    }

    selectedIndex = -1;
    draw();
  });

  canvas.addEventListener('mousemove', function(e) {
    const rect = canvas.getBoundingClientRect();
    const scaleX = canvas.width / rect.width;
    const scaleY = canvas.height / rect.height;
    const mx = (e.clientX - rect.left) * scaleX;
```

```
    const my = (e.clientY - rect.top) * scaleY;

    coordDisplay.textContent = `📍 ${Math.round(mx)}, ${Math.round(my)}`;
    coordDisplay.classList.add('show');

    if (isDragging && dragIndex >= 0 && polygonPixels.length > 0) {
      polygonPixels[dragIndex] = [mx, my];
      draw();
      updateOutput();
      statusEl.textContent = `🖍️ Moving point ${dragIndex}`;
      return;
    }

    if (polygonPixels.length > 0) {
      const idx = findClosestPoint(mx, my);
      if (idx >= 0) {
        canvas.style.cursor = 'grab';
        return;
      }
      const edgeIdx = findClosestPointOnLine(mx, my);
      canvas.style.cursor = edgeIdx >= 0 ? 'pointer' : 'crosshair';
    } else {
      canvas.style.cursor = 'crosshair';
    }
  });

  canvas.addEventListener('mouseup', function() {
    if (isDragging) {
```

```
      isDragging = false;
      canvas.style.cursor = 'crosshair';
      saveHistory();
      statusEl.textContent = ` ✅ Point ${dragIndex} moved`;
      log(` 🖍️ Point ${dragIndex} moved`, 'info');
      dragIndex = -1;
    }
});

canvas.addEventListener('mouseleave', function() {
    if (isDragging) {
      isDragging = false;
      canvas.style.cursor = 'crosshair';
      dragIndex = -1;
    }
    coordDisplay.classList.remove('show');
});

// ---------- Delete point ----------
deletePointBtn.addEventListener('click', function() {
    if (polygonPixels.length <= 3) {
      statusEl.textContent = '⚠️ Need at least 3 points';
      return;
    }
    if (selectedIndex < 0) {
      statusEl.textContent = '⚠️ Click a point to select it first';
      return;
    }
```

```
    saveHistory();
    polygonPixels.splice(selectedIndex, 1);
    selectedIndex = -1;
    draw();
    updateOutput();
    statusEl.textContent = `➖ Deleted point (${polygonPixels.length} total)`;
  });

  // ---------- Undo ----------
  undoBtn.addEventListener('click', undo);

  // ---------- Export SVG ----------
  exportSVGBtn.addEventListener('click', function() {
    if (polygonPixels.length < 3 || !img) {
      statusEl.textContent = '⚠️ No polygon to export';
      return;
    }

    const w = canvas.width;
    const h = canvas.height;
    const worldPoints = convertToWorldUnits(polygonPixels, w, h);
    const label = getUnitLabel();

    // Build SVG points string
    const pointsStr = worldPoints.map(([x, y]) =>
`${x.toFixed(4)},${y.toFixed(4)}`).join(' ');
```

```
// Calculate bounds for SVG viewBox
let minX = Infinity,
  minY = Infinity,
  maxX = -Infinity,
  maxY = -Infinity;
for (const [x, y] of worldPoints) {
  if (x < minX) minX = x;
  if (x > maxX) maxX = x;
  if (y < minY) minY = y;
  if (y > maxY) maxY = y;
}
const width = maxX - minX || 1;
const heightSVG = maxY - minY || 1;
const pad = Math.max(width, heightSVG) * 0.1;

const svgContent = `<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" viewBox="${minX - pad} ${minY - pad} ${width + pad * 2} ${heightSVG + pad * 2}" width="100%" height="100%">
<!-- Golf Green Polygon -->
<polygon points="${pointsStr}" fill="rgba(0,255,136,0.15)" stroke="#00ff88" stroke-width="2" />

<!-- Vertices -->
${worldPoints.map(([x, y]) => `<circle cx="${x.toFixed(4)}" cy="${y.toFixed(4)}" r="3" fill="#00ff88" />`).join('\n  ')}

<!-- Labels -->
${worldPoints.map(([x, y], i) => `<text x="${x.toFixed(4)}" y="${(y - 5).toFixed(4)}" font-size="8" fill="#00ff88" text-anchor="middle">${i}</text>`).join('\n  ')}
```

```
  <text x="${minX - pad + 5}" y="${minY - pad + 15}" font-size="10" fill="#8aaa82" font-family="sans-serif">Green Polygon (${worldPoints.length} vertices) | Units: ${label}</text>
  </svg>`;

    const blob = new Blob([svgContent], { type: 'image/svg+xml' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `green_${Date.now()}.svg`;
    a.click();
    URL.revokeObjectURL(url);

    statusEl.textContent = `📐 SVG exported with ${worldPoints.length} vertices (${label})`;
    log(`📐 SVG exported`, 'success');
  });

  // ---------- Copy ----------
  copyBtn.addEventListener('click', function() {
    if (!outputText.value) return;
    navigator.clipboard.writeText(outputText.value).then(() => {
      copyFeedback.classList.add('show');
      setTimeout(() => copyFeedback.classList.remove('show'), 1500);
      statusEl.textContent = '📋 Copied to clipboard';
    });
  });
```

```
// ---------- Download ----------
downloadBtn.addEventListener('click', function() {
  if (!outputText.value) return;
  const blob = new Blob([outputText.value], { type: 'text/plain' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = `green_xy_${Date.now()}.txt`;
  a.click();
  URL.revokeObjectURL(url);
  statusEl.textContent = '💾 Downloaded';
});

// ---------- Reset ----------
resetBtn.addEventListener('click', function() {
  polygonPixels = [];
  polygonWorld = [];
  history = [];
  selectedIndex = -1;
  outputText.value = '';
  coordCount.textContent = '0 vertices';
  finalCount.textContent = '0';
  areaDisplay.textContent = '0';
  pointCountEl.textContent = 'Points: 0';
  draw();
  statusEl.textContent = 'Reset – upload image and click the green';
  log('↺ Reset', 'info');
});
```

```
// ---------- File input ----------
fileInput.addEventListener('change', function(e) {
  if (e.target.files.length) {
    loadImage(e.target.files[0]);
  }
});

// ---------- Drag and drop ----------
document.addEventListener('dragover', function(e) {
  e.preventDefault();
});

document.addEventListener('drop', function(e) {
  e.preventDefault();
  const files = e.dataTransfer.files;
  if (files.length && files[0].type.startsWith('image/')) {
    loadImage(files[0]);
  }
});

// ---------- Keyboard shortcuts ----------
document.addEventListener('keydown', function(e) {
  if ((e.ctrlKey || e.metaKey) && e.key === 'z') {
    e.preventDefault();
    undo();
  }
  if (e.key === 'Delete' || e.key === 'Backspace') {
```

```
            deletePointBtn.click();
        }
    });

    // ---------- Startup ----------
    log('🚀 XY World Units version ready!', 'success');
    statusEl.textContent = '📩 Upload a satellite image to start';
  </script>
</body>
</html>
```